```
$car = new Car();
$bike = new Motorbike();

print "A car can carry {$car->getMaxPassengers()} people" . PHP_EOL;
print "A motorbike can carry {$bike->getMaxPassengers()} people" . PHP_EOL;

Output:
car can carry 4 people
motorbike can carry 2 people
```

Now we have a `Vehicle` class that contains the common method of `getMaxPassengers()` which all children of `Vehicle` inherit, and all client code makes use of. The problem domain data that was previously controlled inside the `switch/case` block has now become an extendable class property: `protected int $maxPassengers`. Now when we need to add new vehicle types to the solution space, we simply extend `Vehicle` and set the `maxPassengers` property as required.

In this way we have closed `Vehicle` to modification and opened it to extension, and OCP is satisfied (and in doing so our code becomes more robust and easier to maintain). This is OCP by inheritance, which was the original method proposed by Meyer in 1988. Subsequent development of the idea by others (such as Robert Martin in his 1996 paper *The Open-Closed Principle*) introduced a refinement: abstractions (either abstract classes or interfaces) should be used to close the interface to modifications but allow different implementations in the derived classes.

Let's implement OCP for our vehicle problem domain using an `interface` abstraction.

```
interface Vehicle
{
    public function getMaxPassengers(): int;
}

class Train implements Vehicle
{
    public function __construct(private int $carriages)
    {
    }

    public function getMaxPassengers(): int
```